Chapitre 5
LES FICHIERS

Savoirs abordés dans ce chapitre :

S4.1 - Principes de base :

- Flux d'entrée et de sortie de base : fichiers

S4.2 - Algorithmique :

- Manipulations de texte (chaînes de caractères)
- Modèle canonique de gestion d'E/S : ouvrir, lire, écrire, fermer
- Bibliothèque standard (ANSI C)

S4.3 - Structure et gestion des données :

- Formats de fichier : texte (human readable), binaire

S4.4 - Programmation procédurale :

- Manipulation de données ("quoi") en pseudo-langage et/ou en langage C
- Transcription d'algorithmes ("comment") en pseudo-langage et/ou en langage C

Compétences développées dans ce chapitre par rapport aux savoirs abordés :

- C4.4 - Développer un module logiciel (S4 - Développement logiciel)
- C4.5 - Tester et valider un module logiciel (S4 - Développement logiciel)
- C4.6 - Intégrer un module logiciel (option IR) (S4.1, S4.2, S4.3, S4.4)
- C3.2 Analyser et compléter un dossier de spécifications techniques (S4.2 - Algorithmique)

I. GENERALITES

Un fichier est un ensemble d'informations stockées sur une mémoire de masse (disque dur, disquette, bande magnétique, CD-ROM).

En C, un fichier est une suite d'octets. Les informations contenues dans le fichier ne sont pas forcément de même type (un char, un int, une structure ...).Le langage C offre la possibilité de gérer la lecture et l'écriture de données à partir de fichiers.

Lorsqu'un programme doit lire ou écrire des données dans un fichier, le chemin pris par ces données pour arriver à leur destination passe toujours par un tampon (buffer). Ce tampon de fichier est une zone de mémoire RAM dans laquelle sont temporairement stockées avant leur transfert à destination, les données lues ou écrites dans un fichier.

On distingue traditionnellement deux techniques de gestion de fichiers :
- l'accès séquentiel consiste à traiter les informations "séquentiellement", c'est-à-dire dans l'ordre où elles apparaissent (ou apparaîtront) dans le fichier,
- l'accès direct consiste à se placer immédiatement sur l'information souhaitée, sans avoir à parcourir celles qui la précèdent.
La distinction entre accès séquentiel et accès direct n'a plus véritablement de raison d'être. En langage C, vous utiliserez les mêmes fonctions dans les deux cas.

II. MANIPULATION DES FICHIERS

Opérations possibles avec les fichiers: Créer - Ouvrir - Fermer - Lire - Ecrire - Détruire - Renommer. La plupart des fonctions permettant la manipulation des fichiers sont rangées dans la bibliothèque standard STDIO.H.

1 - Déclaration: **FILE *fichier; /* majuscules obligatoires pour FILE */**

On définit un pointeur sur FILE. Ce pointeur fournit l'adresse d'une cellule donnée (l'adresse du tampon).

2 - Ouverture: **FILE *fopen(char *nom, char *mode);**

On passe donc 2 chaînes de caractères
nom: celui figurant sur le disque, exemple: « a :\toto.txt»

mode (pour les fichiers TEXTES) :
- « r » lecture seule
- « w » écriture seule (destruction de l'ancienne version si elle existe)
- « w+ » lecture/écriture (destruction ancienne version si elle existe)
- « r+ » lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version.

« a+ » lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version, le pointeur est positionné à la fin du fichier.

A l'ouverture, le pointeur est positionné au début du fichier (sauf « a+ »)

Exemple : **FILE *fichier ;**
fichier = fopen(« a :\toto.txt », « r+ ») ;

3 - Fermeture: **int fclose(FILE *fichier);**
Retourne 0 si la fermeture s'est bien passée, EOF en cas d'erreur.
Il faut toujours fermer un fichier à la fin d'une session.

Exemple : **FILE *fichier ;**
fichier = fopen(« a :\toto.txt », « r+ ») ;
/* Ici instructions de traitement */
fclose(fichier) ;

4 - Destruction: **int remove(char *nom);**
Retourne 0 si la destruction s'est bien passée.

Exemple : **remove(« a :\toto.txt») ;**

5 - Renommer: **int rename(char *oldname, char *newname);**
Retourne 0 si la fermeture s'est bien passée.

6 - Positionnement du pointeur au début du fichier: **void rewind(FILE *fichier);**

7- Ecriture dans le fichier:

int putc(char c, FILE *fichier);
Ecrit la valeur de c à la position courante du pointeur , le pointeur avance d'une case mémoire.
Retourne EOF en cas d'erreur.

Exemple : **putc('A', fichier) ;**

int putw(int n, FILE *fichier);
Idem, n de type int, le pointeur avance du nombre de cases correspondant à la taille d'un entier (4 cases en C standard).
Retourne n si l'écriture s'est bien passée.

int fputs(char *chaîne, FILE *fichier); idem avec une chaîne de caractères, le pointeur avance de la longueur de la chaine ('\0' n'est pas rangé dans le fichier).
Retourne EOF en cas d'erreur.

Exemple : **fputs(« BONJOUR ! », fichier) ;**

int fprintf(FILE *fichier, char *format, liste d'expressions);
Retourne EOF en cas d'erreur.

Exemples: **fprintf(fichier,"%s","il fait beau");**
fprintf(fichier,%d,n);
fprintf(fichier,"%s%d","il fait beau",n);

Le pointeur avance d'autant.

8 - Lecture du fichier:

int getc(FILE *fichier); lit 1 caractère, mais retourne un entier n; retourne EOF si erreur ou fin de fichier; le pointeur avance d'une case.
Exemple: **char c ;**
c = (char)getc(fichier) ;

int getw(FILE *fichier); idem avec un entier; le pointeur avance de la taille d'un entier.
Exemple: **int n ;**
n = getw(fichier) ;

char *fgets(char *chaine,int n,FILE *fichier); lit n-1 caractères à partir de la position du pointeur et les range dans chaine en ajoutant '\0'.

int fscanf(FILE *fichier, char *format, liste d'adresses); analogue à fprintf en lecture.
Exemple : fscanf(fichier, « %d »,&nb) ;

9 - Gestion des erreurs:

fopen retourne le pointeur NULL si erreur (Exemple: impossibilité d'ouvrir le fichier).

fgets retourne le pointeur NULL en cas d'erreur ou si la fin du fichier est atteinte.

la fonction **int feof(FILE *fichier)** retourne 0 tant que la fin du fichier n'est pas atteinte.

III) Accès direct

On dit d'un fichier qu'il est utilisé en **accès direct** s'il est possible de lire ou d'écrire des infomations dedans dans un ordre quelconque.

En C, la distinction n'est pas faite sur les fichiers, on suppose que le système d'exploitation est capable de fournir les modes d'accès séquentiel et direct. Ce sont les **fonctions** qui font la différence entre accès direct et séquentiel.

Toutes les fonctions que nous avons vu jusqu'ici travaillent en "mode séquentiel", elles avancent du nombre de caractères qu'elles ont pu lire ou écrire.
Les deux fonctions suivantes permettent à tout moment de se positionner n'importe où dans le programme et de connaître la position actuelle.

fseek()
int fseek(FILE * stream, long offset, int method);

La fonction **fseek()** permet de déplacer la position courante de lecture ou d'écriture.
L'argument **distance r**eprésente le déplacement (positif ou négatif) en
octets à effectuer dans le fichier **strea**m. Il faut se méfier avec ce paramètre car il doit être de type **lon**g.
Trois méthodes de déplacement sont à disposition. Ce choix est fixé par le paramètre **metho**d; suivant sa valeur le déplacement aura lieu :
0 (ou la constante prédéfinie **SEEK_SET**) : par rapport au début du fichier
1 (SEEK_CUR) : par rapport à l'emplacement courant
2 (SEEK_END) : par rapport à la fin du fichier
La fonction **fseek()** retourne une valeur non nulle (vraie) si le déplacement n'a pu avoir lieu, **0** dans le cas contraire.
A remarquer que **SEEK_END** et une distance négative a pour effet de reculer depuis la fin.

Ainsi, l'appel **rewind(fp)** est identique à **fseek(fp, 0L, 0).**

ftell()
long ftell(FILE * stream); /* ANSI, prototype dans stdio.h */

La fonction **ftell()** retourne la position courante de lecture/écriture du fichier
Stream mesurée en octets depuis le début du fichier. Cette fonction est utile pour mémoriser une position afin de s'y repositionner ultérieurement.
Ainsi, si on désire trouver la longueur d'un fichier, on peut le faire facilement par les deux instructions

```
FILE *fich;
int longueur;
fseek (fich, 0L, 2); /* aller après le dernier byte du fichier */
longueur = ftell(fich); /* numéro du dernier byte */
```

<u>à voir en TP</u>

fread()
/* ANSI, prototype dans stdio.h */
size_t fread(void * ptr, size_t size, size_t n, FILE * stream);
int fread(**pt**r, **siz**e, n, **strea**m) /* declaration dans stdio.h */

char * ptr;
unsigned size;
int n;
FILE * stream;
Cette fonction lit depuis le flot **strea**m un nombre d'informations n dont chacune est de
taille **siz**e. Le résultat de cette lecture est stocké à partir de l'adresse **pt**r.
La fonction retourne le nombre d'informations lues si aucune erreur n'est intervenue. Ce
nombre peut être inférieur au nombre nsi la fin de fichier a été rencontrée ou lorsqu'une
erreur intervient.
Les fonctions **feof()** et **ferror()** peuvent être employées pour distinguer une erreur d'une fin de fichier lorsque la valeur retournée est inférieure à n.

fwrite()
/* ANSI, prototype dans stdio.h */
size_t fwrite(const void * ptr,size_t size,size_t n,FILE * stream);
int fwrite(**pt**r, **siz**e, n, **strea**m) /* declaration dans stdio.h */
char * ptr;
unsigned size;
int n;
FILE * stream;
fwrite() transfère, sur le flot **strea**m, ninformations de taille **siz**estockées à partir de l'adresse **pt**r.
La fonction retourne le nombre d'informations écrites, une erreur a eu lieu si ce nombre
est inférieur à n.
Ces deux fonctions s'utilisent en général avec les fonctions de déplacement décrites en
page précédente pour effectuer des entrées/sorties d'enregistrements en accès direct.